UCAM

SECCIÓN 1.4

EVOLUCIÓN DEL ROL DBA

De la administración artesanal a la ingeniería de confiabilidad de plataformas.

1.4.1 Gobierno de Datos, Operación y Continuidad Estratégica

La **nuevo DBA** no solo gestiona "***sistemas que guardan datos***"; **gobierna infraestructuras que garantizan la viabilidad operativa, legal y financiera del negocio**.

La pregunta clave evoluciona: ya no es qué hace el sistema, sino

- **Quién orquesta su resiliencia:** capacidad de sobrevivir a fallos sin perder datos.
- **Evolución a gran escala:** facultad de crecer en capacidad, usuarios y complejidad de forma sostenible en el tiempo

Resiliencia	Evolución a gran escala
• Sobrevivir a fallos sin perder datos	• Crecer en capacidad, • usuarios y • complejidad

Responsabilidades Operativas

1. Operaciones	El **despliegue de entornos:** configuración inicial, el parcheo y actualización, migraciones entre versiones o plataformas, **monitoreo** básico.
2. Capacity Planning Predictivo	**Modelar matemáticamente el crecimiento transaccional para provisionar recursos** (CPU, memoria, IOPS) **meses antes de que el motor sufra inanición**. *IOPS (Input/Output Operations Per Second)* se refiere al **ancho de banda** del disco con peticiones simultáneas de Lectura/Escritura; si no tiene suficiente RAM, aumentan las IOPS, CPU espera a que responda el disco, se acumulan consultas en cola y aumenta dramáticamente la latencia. Un sistema bien administrado **no reacciona únicamente a la crisis; intenta preverla**. Esto es una **señal** temprana de **pensamiento estratégico.**

1.4.1 Gobierno de Datos, Operación y Continuidad Estratégica

Responsabilidades de Seguridad, Auditoría y Tuning

3. Seguridad	**Garantizar el principio de menor privilegio**, **trazar cada mutación de esquema** (DDL: CREATE, ALTER, DROP, CONSTRAINS) para saber quien aplicó una migración, qué columna alteró y cuándo lo hizo. **Garantiza que los accesos sean apropiados**. Un usuario de aplicación solo debe tener permisos para manipular datos (DML), nunca para cambiar esquemas (DDL).
4. Auditoría y Gobierno	***Auditar y monitoreo en tiempo real*** *(motores de auditoría, alertas automa.)****, encriptación en reposo*** *(archivos físicos y copias de seguridad)* ***y en tránsito*** *(protocolos de comunicación cifrada)*. Garantizar que exista ***trazabilidad sobre acciones críticas*** *(logs de auditoría y cumplimento de normativas legales GDPR, ISO 27001, PCI-DSS).*
5. Tuning	Ejecuta un ***tuning continuo del motor*** (RAM, mem. Caché, hilos, parámetros I/O), ***query profiling*** (instrumentar, medir y analizar la ejecución sentencias SQL para evaluar su lentitud o consumo excesivo de recursos) y ***Lock Management*** (detectar transacciones que bloquean otras transacciones, eliminar deadlocks, ajustar estrategias de concurrencia y paralelismo), **optimizando planes de ejecución** (analizar rutas que elige el motor para resolver consultas) y ***estructuras físicas*** (crear, modificar o eliminar índices, particiones de tablas masivas, reorganizar almacenamiento) antes de que impacten la latencia de las aplicaciones.

Continuidad Estratégica y Recuperación Verificada

6. Continuidad	Diseñar e implementar un ciclo de vida automatizado de prueba, validación y recuperación de copias de seguridad: Restauración automatizada en aislamiento, verificación de integridad física y lógica, almacenamiento (regla 3-2-1)
7. Recuperabilidad verificada	Un DBA no asume la recuperabilidad; diseña, automatiza y audita simulacros de ***Disaster Recovery*** (DR) midiendo métricas estrictas como RPO (Punto de Recuperación) y RTO (Tiempo de Recuperación).

1.4.2 Infraestructura como código (IaC) y Automatización

El paradigma moderno exige superar la **administración manual tradicional** —como instalar y configurar un motor directamente sobre un sistema operativo local— para adoptar **flujos de trabajo declarativos**.

Flujos de trabajo

Despliegues Reproducibles	**Toda la configuración** (cuántos nodos, qué red usan, cuánta memoria tienen) **se escribe en un archivo de texto** (código). **Herramientas** como Terraform o Kubernetes **leen ese archivo y levantan la infraestructura automáticamente**. Si el servidor cae, **ejecutas el código y tienes un clon exacto funcionando de inmediato.**
Reducción del Riesgo Humano	El mayor peligro para una base de datos es un humano tecleando un comando con sueño o por error en el servidor equivocado. Ya nadie hace cambios directamente en producción. **Los cambios se envían a un sistema automático** (tuberías CI/CD) **que los prueba primero en entornos seguros**. Los **cambios se integran mediante tuberías CI/CD**, permitiendo que la reconstrucción de un clúster caído tome minutos en lugar de días, garantizando una **trazabilidad forense** absoluta. Como todo cambio es un archivo de código guardado en un historial (como Git), si algo se rompe, el equipo sabe exactamente quién hizo el cambio, qué línea modificó y cuándo, lo que permite dar marcha atrás con un clic.
Convergencia de Roles	Al pasar a gestionar la base de datos con código y tuberías automáticas, **el trabajo del DBA se fusiona con el del Ingeniero de Plataforma y el SRE** (Site Reliability Engineer). Ahora la base de datos se administra con las mismas herramientas de ingeniería de software modernas que el resto de las aplicaciones de la empresa.

1.4.3 Anatomía del perfil T-Shaped de Alto Nivel

Un ingeniero de datos de élite (ideal) no se define por memorizar comandos de una herramienta específica, sino por su **capacidad sistémica** modelada en forma de "T":

T-Shaped

Eje vertical Especialización Profunda	Dominio absoluto de los entresijos del motor. Comprender íntimamente la **gestión de memoria** (buffer pools), los **bloqueos de concurrencia**, las estructuras de **índices** (Árboles B+) y la escritura en **logs** transaccionales.
Eje horizontal Conocimiento Transversal	Comprensión avanzada de los **ecosistemas que rodean al dato**: • kernel de sistemas operativos, • redes, • lenguajes de programación, • orquestación y • almacenamiento distribuido.
Visión Estratégica y Comunicación	La capacidad innegociable de **traducir problemas técnicos** (mala arquitectura) **en riesgos financieros medibles**, justificando decisiones técnicas ante la alta dirección con métricas y evidencias empíricas (Ej. "Si no optimizamos esta base de datos, los tiempos de carga nos harán perder un 15% de ventas online durante el mes").

UCAM

SECCIÓN 1.5

Panorama de tecnologías de BD

Familias de motores, cargas de trabajo y despliegues contemporáneos.

1.5.1 Familias de motores: Relacionales y Ecosistemas NoSQL

Cada paradigma de persistencia y acceso ha sido **diseñado para resolver problemas estructurales específicos**.

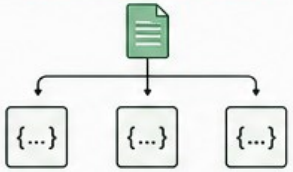
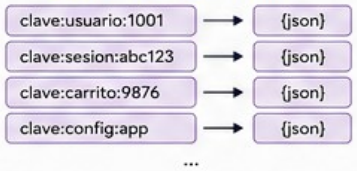
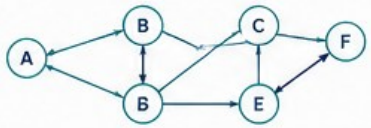
No se elige una familia por tradición, sino evaluando la **topología de la carga de trabajo** (lectura intensiva vs. escritura intensiva), la volatilidad del esquema y los requisitos de consistencia.

Familia	Paradigma Estructural	Carga de Trabajo Óptima
Relacional (RDBMS)	Estructuras tabulares fuertemente tipadas, **álgebra relacional** y cumplimiento estricto de propiedades **ACID**.	**Sistemas transaccionales críticos** (Core bancario, ERPs), donde la integridad referencial y la consistencia absoluta son innegociables.
Documental (NoSQL)	Esquemas flexibles (*schema-on-read*). Los datos se guardan en **documentos jerárquicos autodescriptivos** (JSON/BSON).	Catálogos de **comercio electrónico, gestión de contenidos** (CMS) o perfiles de usuario con atributos muy variables.
Clave-Valor (NoSQL)	**Diccionarios masivos y particionables**. Acceso de complejidad algorítmica O(1) directo a un dato a través de su identificador.	**Cachés de alto rendimiento, gestión de sesiones web y carritos de la compra temporales**.
Columna (NoSQL)	Los datos se almacenan físicamente **agrupando columnas** en lugar de filas, optimizando la **compresión** y la **lectura parcial**.	**Analítica masiva** (OLAP), **almacenes de datos** (*Data Warehouses*) y recolección de **series temporales** o **telemetría IoT**.
Grafos (NoSQL)	Basado en **nodos**, **aristas** (relaciones) y **propiedades.** Elimina las costosas penalizaciones de los *JOINs* múltiples.	**Detección de fraudes** en tiempo real, **motores de recomendación** complejos y **análisis de redes sociales**.

1.5.1 FAMILIAS DE MOTORES: RELACIONALES Y ECOSISTEMAS NoSQL

UCAM

Cada paradigma de persistencia y acceso ha sido diseñado para resolver problemas estructurales específicos.
No se elige una familia por tradición, sino evaluando la **topología de la carga de trabajo** (lectura intensiva vs. escritura intensiva), la volatilidad del esquema y los requisitos de consistencia.

FAMILIA	PARADIGMA ESTRUCTURAL	¿CÓMO ORGANIZA LOS DATOS?	CARGA DE TRABAJO ÓPTIMA	EJEMPLOS DE USO	MOTORES COMERCIALES LÍDERES
Relacional (RDBMS)	Estructuras tabulares fuertemente tipadas, **álgebra relacional** y cumplimiento estricto de propiedades **ACID**.	**Tablas relacionadas** 	Sistemas **transaccionales críticos** (Core bancario, ERPs), donde la integridad referencial y la consistencia absoluta son innegociables.	• Bancos y finanzas (ACID) • ERPs y sistemas contables • Gestión de inventarios • Reservas y ticketing • Sistemas gubernamentales	ORACLE Database Microsoft SQL Server IBM Db2
Documental (NoSQL)	Esquemas flexibles (*schema-on-read*). Los datos se guardan en **documentos jerárquicos autodescriptivos** (JSON/BSON).	**Documentos JSON/BSON** 	Catálogos de **comercio electrónico, gestión de contenidos** (CMS) o perfiles de usuario con atributos muy variables.	• Catálogos de productos • CMS y gestión de contenidos • Perfiles de usuario (web/app) • Logs y eventos • Aplicaciones móviles	mongoDB. Couchbase MarkLogic
Clave-Valor (NoSQL)	Diccionarios masivos y particionables. Acceso de complejidad algorítmica **O(1)** directo a un dato a través de su **identificador**.	**Estructura Clave → Valor** 	Cachés de **alto rendimiento**, gestión de **sesiones web** y **carritos** de la compra temporales.	• Caché de alto rendimiento • Sesiones de usuario • Carritos de compra • Contadores y rankings • Configuración de aplicaciones	redis amazon DynamoDB AEROSPIKE
Columna (NoSQL)	Los datos se almacenan físicamente **agrupando columnas** en lugar de filas, optimizando la **compresión** y la **lectura parcial**.	**Almacenamiento por Columnas** 	**Analítica masiva (OLAP)**, **almacenes de datos** (*Data Warehouses*) y recolección de **series temporales** o **telemetría IoT**.	• Data Warehousing / OLAP • Business Intelligence • Series temporales (IoT) • Telemetría y métricas • Exploración de grandes volúmenes	ClickHouse Google Bigtable Amazon Redshift
Grafos (NoSQL)	Basado en **nodos, aristas** (relaciones) y **propiedades**. Elimina las costosas penalizaciones de los **JOINs múltiples**.	**Nodos (Vértices) y Aristas (Relaciones)** 	Detección de **fraudes** en tiempo real, motores de **recomendación** complejos y análisis de **redes sociales**.	• Detección de fraude • Recomendaciones • Análisis de redes sociales • Rutas y logística • Gestión de identidades y accesos	neo4j JanusGraph Amazon Neptune

1.5.2 Topología de Cargas de Trabajo: OLTP, OLAP y HTAP

OLTP vs OLAP

El ingeniero de datos debe clasificar el tráfico del sistema en categorías operacionales precisas para modelar, indexar y configurar los recursos de hardware en consecuencia.

La distinción más profunda y con mayor valor formativo se encuentra entre el procesamiento transaccional y el analítico.

Entender estas diferencias explica por qué las estructuras de índices, los algoritmos de caché y la configuración de memoria difieren radicalmente según el caso de uso.

Dimensión Arquitectónica	OLTP (Procesamiento Transaccional en Línea)	OLAP (Procesamiento Analítico en Línea)
Naturaleza de la Carga	**Micro-transacciones** de alta frecuencia y baja latencia (Inserciones, actualizaciones y borrados rápidos).	**Consultas masivas** de lectura que escanean millones de registros para agrupar y totalizar datos históricos.
Métrica de Rendimiento	**IOPS** (Operaciones de Entrada/Salida por Segundo) y latencia por transacción.	**Throughput** (Ancho de banda de lectura) y tiempo total de ejecución de la consulta.
Modelado de Datos	**Alta normalización** (3NF) para evitar redundancias y anomalías de actualización.	**Desnormalización intencionada** (Esquemas de Estrella o Copo de Nieve) para evitar múltiples JOINs costosos.
Almacenamiento Físico	**Orientado a filas**. Se lee o escribe el registro completo de una sola vez.	**Orientado a columnas** (en motores especializados) para comprimir y leer solo los atributos necesarios.

1.5.2 TOPOLOGÍA DE CARGAS DE TRABAJO: OLTP, OLAP y HTAP

Clasifique su carga, elija la arquitectura correcta y alinee su motor de base de datos con los objetivos del negocio.

DIMENSIÓN ARQUITECTÓNICA	OLTP (PROCESAMIENTO TRANSACCIONAL EN LÍNEA)	OLAP (PROCESAMIENTO ANALÍTICO EN LÍNEA)	HTAP (HYBRID TRANSACTIONAL / ANALYTICAL PROCESSING)
Naturaleza de la Carga	**Micro-transacciones** de alta frecuencia y baja latencia. Inserciones, actualizaciones y borrados rápidos.	**Consultas masivas** de lectura que escanean millones de registros para agrupar, resumir y descubrir patrones históricos.	Mezcla simultánea de transacciones y analítica. Análisis operacionales en tiempo real sin mover datos.
Métrica de Rendimiento	**IOPS** (Operaciones de Entrada/Salida por Segundo) y latencia por transacción (ms).	**Throughput** (ancho de banda de lectura) y tiempo total de ejecución de la consulta.	Balance entre **IOPS** (escrituras) y **Throughput** (lecturas) con latencia predecible para ambos.
Modelado de Datos	**Alta normalización** (3NF) para garantizar integridad referencial y evitar redundancias y anomalías de actualización.	**Desnormalización intencionada** (Esquemas de Estrella o Copo de Nieve) para reducir JOINs costosos y acelerar agregaciones.	**Modelado flexible** (estrella + dimensiones actualizadas) que soporta escrituras normalizadas y vistas analíticas pre-agregadas.
Almacenamiento Físico	**Orientado a filas (Row Store).** Se lee o escribe el registro completo de una sola vez.	**Orientado a columnas (Column Store).** Lee solo las columnas necesarias, ideal para compresión y lectura parcial.	**Almacenamiento dual:** filas para transacciones + columnas para analítica (sin duplicación de datos en disco en motores nativos).
Índices Predominantes	**Árbol B+** (altamente selectivos). Índices únicos, compuestos y particionados.	**Índices de mapa de bits**, índices de bitmap join, diccionarios y zonas de ordenamiento.	**Índices adaptativos:** B+ Trees + estructuras columnar y skip indexes.
Características Clave	• ACID estricto • Bloqueos de grano fino (fila) • Alta concurrencia de escrituras • Latencia sub-milisegundo	• Lectura intensiva • Agregaciones complejas • Escaneo de grandes volúmenes • Tolerancia a latencia (segundos/minutos)	• Analítica en tiempo real • Eliminación del retraso ETL • Aislamiento de recursos • Consistencia operativa y analítica
Ejemplos de Uso	• Core bancario y tarjetas de crédito • Sistemas de reservas y ticketing • ERPs (SAP, Oracle E-Business Suite) • Gestión de inventarios en tiempo real	• Data Warehouses corporativos • Business Intelligence y reporting • Análisis de ventas históricas • Data Mining y segmentación	• Detección de fraude en tiempo real • Recomendaciones personalizadas al instante • Analítica operativa (dashboards en vivo) • IoT y telemetría industrial
Motores Comerciales Líderes	ORACLE Database, Microsoft SQL Server, IBM Db2, PostgreSQL, MySQL	amazon REDSHIFT, Google BigQuery, ClickHouse, snowflake, SAP HANA	SingleStore, TiDB, Cockroach Labs, MariaDB ColumnStore, Oracle Autonomous Database

1.5.2 Topología de Cargas de Trabajo: OLTP, OLAP y HTAP

Un **sistema bancario** o un carrito de compras (OLTP) exige un bloqueo de registros minucioso y una latencia mínima. Ejecutar un motor DB de forma nativa en un servidor, sin las capas de red virtualizadas que añade Docker, se obtiene un acceso mucho más directo al sistema de archivos del sistema operativo.

Un **sistema de reporting directivo** (OLAP) asfixiaría a ese mismo servidor OLTP, ya que un informe de ventas anuales bloquearía las tablas durante minutos, impidiendo que entren nuevas ventas.

HTAP

Tradicionalmente, para evitar que el análisis masivo (OLAP) colapsara el sistema transaccional (OLTP), las organizaciones utilizaban costosos procesos **ETL** (Extracción, Transformación y Carga) nocturnos para mover los datos desde la base de datos principal hacia un Data Warehouse separado.

Actualmente se diseñan arquitecturas **HTAP** (Hybrid Transactional/Analytical Processing). Este paradigma **busca eliminar el retraso de los ETL y ofrecer analítica operacional en tiempo real**.

Los sistemas HTAP logran esto mediante innovaciones profundas a nivel del motor:

- ✓ **Motores de almacenamiento duales**: Mantienen los datos en memoria en formato de filas para las transacciones (OLTP) y, simultáneamente de forma transparente, replican esa memoria en formato columnar para las consultas analíticas (OLAP).
- ✓ **Aislamiento de recursos**: Garantizan que un análisis pesado sobre los datos en tiempo real no robe ciclos de CPU a las operaciones de inserción críticas. Esta comprensión evita que el administrador configure el motor "a ciegas", permitiéndole alinear la física del sistema de datos con la realidad de las reglas de negocio.

1.5.2 TOPOLOGÍA DE CARGAS DE TRABAJO: OLTP, OLAP y HTAP

Clasifique su carga, elija la arquitectura correcta y alinee su motor de base de datos con los objetivos del negocio.

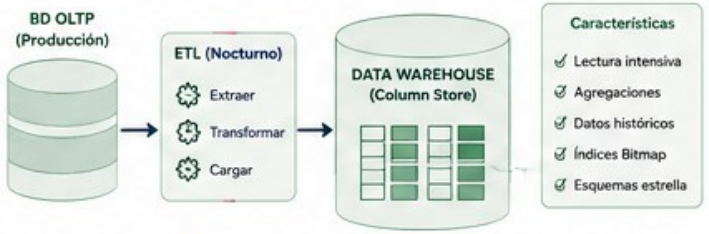
FLUJO DE DATOS Y ARQUITECTURAS TÍPICAS

OLTP: SISTEMA TRANSACCIONAL



Objetivo: Procesar transacciones rápido y de forma confiable.

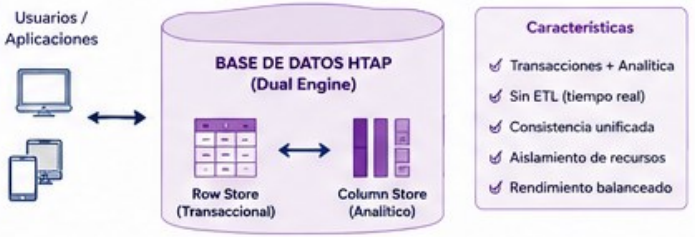
OLAP: SISTEMA ANALÍTICO (TRADICIONAL)



Objetivo: Analizar grandes volúmenes sin afectar el OLTP.

RETRASO ETL: Horas o días

HTAP: ARQUITECTURA UNIFICADA (TIEMPO REAL)



Objetivo: Analítica operativa en tiempo real sobre datos actuales.

TIEMPO REAL

¿CÓMO DECIDIR?

- Si su prioridad es **procesar transacciones rápidas y consitentes** → **OLTP**
- Si su prioridad es analizar grandes volúmenes **históricos sin afectar** producción → **OLAP**
- Si necesita analizar y actuar sobre datos **actuales al instante** → **HTAP**

La elección correcta alinea la arquitectura con el objetivo del negocio y evita cuellos de botella costosos.

BUENAS PRÁCTICAS PARA CADA TOPOLOGÍA

PARA OLTP

- Diseñar índices selectivos
- Mantener transacciones cortas
- Monitorear IOPS y latencia
- Evitar bloqueos de larga duración

PARA OLAP

- Usar particionamiento y compresión
- Diseñar esquemas estrella
- Cargar datos en lotes (ETL eficiente)
- Aprovechar índices de mapa de bits

PARA HTAP

- Usar motores nativos HTAP
- Configurar aislamiento de recursos
- Monitorear balance escrituras/lecturas
- Diseñar modelos flexibles

LEYENDA DE ICONOS



1.5.3 Topología de despliegue: On Premise, cloud y distribuidos

El ingeniero de datos/DBA debe clasificar el tráfico del sistema en categorías operacionales precisas para modelar, indexar y configurar los recursos de hardware en consecuencia.

Para un arquitecto o un DBA de élite, estas no son simples "opciones de instalación", sino posturas estratégicas frente al riesgo, el coste y el control:

On-Premise (Soberanía y Bare-Metal)

La organización **controla desde el hormigón del centro de datos hasta la aplicación**. Aunque se percibe como tradicional, es la **opción elegida para cargas que exigen microsegundos de latencia** (ej. High-Frequency Trading), **control absoluto del hardware** (kernel tuning) o **cumplimiento normativo estricto** (aislamiento air-gapped).

Cloud (Agilidad y Elasticidad): *Consumo de recursos bajo demanda*

Transforma el ***gasto de capital*** (CapEx) en ***gasto operativo*** (OpEx). Exige un diseño orientado a ***lidiar con la latencia de red*** inherente a los centros de datos públicos y la ***adopción de prácticas de FinOps*** para evitar el despilfarro de recursos.

Sistemas Geo-Distribuidos

El **nivel más complejo**. La base de datos se extiende por **múltiples zonas geográficas**. Aquí, la ***coordinación entre nodos***, la ***resolución de conflictos*** (split-brain) y la ***latencia dictada por la velocidad de la luz*** se vuelven los principales desafíos matemáticos y operativos del diseño.

ARQUITECTURAS DE BASES DE DATOS MODERNAS (2026)

Tres enfoques arquitectónicos para distintos objetivos de rendimiento, escalabilidad y resiliencia.

UCAM

1. ON-PREMISE / BARE-METAL

Centro de Datos / Hardware Físico
Servidores dedicados, almacenamiento local red de baja latencia

Sistema Operativo Base
Control total del sistema operativo, parches, configuración y recursos

Motor de BD Nativo
Instalación nativa, acceso directo al hardware, sin capa de virtualización

VENTAJA
Microsegundos de latencia
Control absoluto del SO y del hardware

CASOS IDEALES
- Máximo rendimiento
- Datos sensibles / regulados
- Workloads predecibles
- Inversiones CapEx justificados

2. CLOUD / DBaaS

Proveedor de Nube Capa Física Oculta
Infraestructura compartida, almacenamiento distribuido, redes de alta capacidad

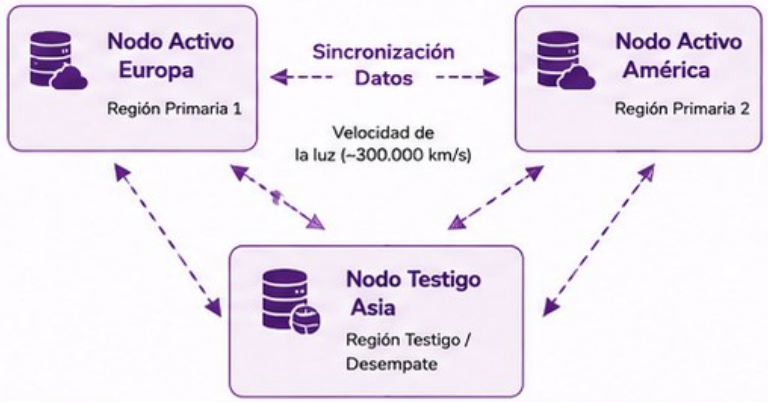
Hipervisor / Virtualización
Aislamiento, asignación dinámica de recursos, multitenencia

Instancia de BD Elástica
Aprovisionamiento bajo demanda, escalado vertical y horizontal, copias automáticas

VENTAJA
Agilidad y escalabilidad (OpEx)
Reto: Latencia del Data Center

CASOS IDEALES
- Agilidad y time-to-market
- Cargas variables / impredecibles
- Escalado elástico
- Modelo operativo OpEx

3. SISTEMAS GEO-DISTRIBUIDOS



VENTAJA
Resiliencia Global y Alta Disponibilidad
Reto: Resolución de Conflictos y Consistencia

CASOS IDEALES
- Alta disponibilidad global
- Tolerancia a desastres
- Experiencia local (baja latencia)
- Negocio 24/7 global

COMPARATIVA CLAVE

	Latencia	Control	Escalabilidad	Modelo Económico	Complejidad	Resiliencia	Casos de Uso Típicos
ON-PREMISE Bare-Metal	Muy baja (microsegundos)	Máximo (SO + Hardware)	Limitada por hardware (Escalado vertical)	CapEx (Inversión inicial alta)	Media (Gestión propia)	Limitada a sitio local	Transacciones críticas, datos sensibles, cumplimiento estricto
CLOUD / DBaaS	Media-Alta (milisegundos)	Compartido (Proveedor)	Alta (Vertical y horizontal)	OpEx (Pago por uso)	Baja-Media (Gestionado)	Alta (Regiones)	Aplicaciones modernas, cargas variables, startups, innovación rápida
GEO-DISTRIBUIDOS	Variable (Depende de distancia)	Compartido + Distribuido	Muy Alta (Distribuida globalmente)	OpEx / Híbrido (Complejo)	Alta (Arquitectura compleja)	Muy Alta (Multi-región)	Negocios globales, alta disponibilidad, continuidad de negocio

LEYENDA DE SÍMBOLOS
Infraestructura Física · Software / Sistema · Base de Datos · Servicios en la Nube · Red / Distribución · Ventajas / Retos · Seguridad / Control · Económico · Rendimiento · Complejidad

1.5.3 Topología de despliegue: On Premise, cloud y distribuidos

Migrar a la nube no es un movimiento binario; es ***elegir en qué punto del espectro de abstracción decide operar el equipo de ingeniería***. Cada modelo redefine la **frontera entre lo que gestiona el proveedor y lo que administra el equipo interno.**

Modelo de Despliegue	Control del Proveedor Cloud	Responsabilidad del DBA
IaaS (Infraestructura como Servicio)	Red física, almacenamiento en bloque y virtualización (Hipervisor).	Sistema operativo, parcheo, instalación del motor, configuración de almacenamiento y arquitectura de alta disponibilidad (HA).
PaaS / DBaaS (Base de Datos como Servicio)	Hardware, SO, parcheo del motor, aprovisionamiento y, en ocasiones, backups automatizados.	Modelado lógico, diseño de índices, optimización de consultas (tuning), control de accesos y capacity planning a nivel de rendimiento.

En un **modelo DBaaS no elimina la necesidad de un administrador**.

La nube proporciona infraestructura resiliente, pero la **responsabilidad sobre el dato siempre es del cliente**.

1.5.4 Tendencias: IA y Semántica

Bases de Datos Autónomas: Gobernanza sobre operación

Utilizando **modelos de inteligencia artificial integrados directamente en el núcleo del SGBD**, el sistema asume de forma nativa la gestión mecánica de la infraestructura subyacente.

Este avance **no reemplaza al ingeniero de datos**, sino que transforma radicalmente sus responsabilidades técnicas:

Optimización Dinámica (Auto-Tuning)

El motor ***evalúa estadísticamente los planes de ejecución en tiempo real, creando o eliminando estructuras físicas*** (como índices) basándose en la heurística del tráfico, ***sin requerir ventanas de mantenimiento manuales***.

Resiliencia y Parcheo (Self-Healing)

Aplicación automática de actualizaciones de seguridad críticas y escalado de recursos computacionales ***en caliente***, lo que mitiga vulnerabilidades sin interrumpir el servicio.

Elevación del Rol

El administrador delega el ***micro-mantenimiento para enfocarse en la estrategia arquitectónica, el cumplimiento normativo, el control financiero*** (FinOps) y el ***diseño de políticas de gobernanza***.

1.5.4 Tendencias: IA y Semántica

Bases de Datos Vectoriales e IA Generativa

Para integrar el conocimiento privado de una empresa en herramientas de **IA Generativa mediante arquitecturas RAG** (Retrieval-Augmented Generation), emergen los **motores vectoriales**.

Representación en Embeddings

Textos documentales, imágenes o código fuente ***se transforman mediante redes neuronales en densos vectores*** numéricos dentro de un ***espacio de alta dimensionalidad***.

Búsqueda por similitud matemática

En lugar de buscar coincidencias exactas con operadores tradicionales, el motor recupera los registros más relevantes midiendo la ***distancia geométrica*** entre los datos (utilizando algoritmos matemáticos como la ***similitud del coseno*** o ***k-NN***).

Expansión del Paradigma Analítico

La base de datos deja de ser exclusivamente un registro de transacciones tabulares para ***convertirse en un sistema capaz de recuperar información basada en su significado y semántica profunda***.

UCAM

SECCIÓN 1.6

Introducción a la Ingeniería Profesional

La adopción de un método científico, ético y reproducible para la gestión del dato.

1.6.1 La Cultura de la Evidencia y Línea Base Operativa

La excelencia en la administración de bases de datos trasciende el dominio técnico del motor; exige adoptar la disciplina operativa de las empresas tecnológicas de primer nivel, inspirada en la **Ingeniería de Confiabilidad del Sitio** (SRE).

Este bloque metodológico define el estándar de trabajo del curso:

- **No se ejecutan instrucciones a ciegas, se diseña una trazabilidad absoluta**. (en git script, en SQLPLUS SPOOL)
- **Lo que no puede auditarse y reproducirse, no es apto para un entorno de producción**.

En infraestructuras críticas, intervenir un sistema sin haber establecido una línea base es el equivalente a **operar a ciegas**.

La **captura sistemática de evidencias** asegura que **cualquier cambio es justificable, medible y, sobre todo, reversible**.

Captura del Estado Inicial (Baseline)

Antes de teclear el primer comando, se deben **recolectar las métricas de consumo y los registros de error**.

```
$ docker stats --no-stream oralab-23ai
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O     PIDS
5e18c60af127   oralab-23ai   2.41%     2.096GiB / 15.45GiB   13.57%    5.07MB / 5.08MB   25GB / 30GB   94
```

Si el motor se ejecuta de forma nativa directamente sobre el sistema operativo, sin contenedores de por medio, la evidencia debe incluir obligatoriamente los **logs del kernel** y el **uso de CPU/RAM** del anfitrión, ya que las **anomalías del sistema impactarán directamente en el rendimiento de la base de datos**.

```
$ free -h > 01_baseline_recursos.txt                    # 1. Capturar la memoria RAM del servidor anfitrión
$ top -b -n 1 | head -n 15 >> 01_baseline_recursos.txt  # 2. Capturar los procesos que más CPU consumen
```

Telemetría Activa

Consiste en **registrar las latencias**, el estado de los **bloqueos y los planes de ejecución** actuales para poder **contrastar matemáticamente el impacto de la intervención** (el "antes y el después").

1.6.2 Documentación como Código: Markdown y GitOps

La **bitácora técnica** deja de ser un simple diario escolar para convertirse en una **Infraestructura del Conocimiento**.

Adoptando las prácticas de la industria, **la documentación se elabora con el mismo rigor que el código fuente**:

Inmutabilidad Criptográfica

El uso de Markdown gestionado a través de repositorios Git garantiza un **historial inmutable** (código criptográfico único del commit; deja rastro) **de cada decisión arquitectónica**, **comando ejecutado, salida de error y solución aplicada**.

Actúa como la caja negra de un avión. Si un sistema falla, puedes revisar el historial en Git y ver exactamente qué comandos ejecutó el ingeniero hace meses, qué errores le salieron y cómo lo solucionó, garantizando que el historial es 100% real y auditable.

Reproducibilidad Estricta

La documentación debe ser tan exacta, secuencial y completa que funcione como una receta matemática.

El objetivo de esta bitácora es que cualquier otro ingeniero (en otro continente y uso horario), pueda **clonar el repositorio y reconstruir el servidor o solucionar la caída del sistema** leyendo las instrucciones paso a paso, basándose únicamente en el texto, sin depender de hablar con la persona que lo escribió originalmente, del conocimiento implícito o de reuniones de traspaso.

Documentar no es tomar "apuntes" para ti mismo, sino **construir una infraestructura de conocimiento sólida, auditable** (Git) y **universalmente comprensible** (Markdown) **para tu equipo**.

1.6.2 Documentación como Código: Markdown y GitOps

- Fase 1: Inicio del Repositorio y Trazabilidad del Sistema (dir, git init, script). Se crea README.md o informe_intervención.md
- Fase 2: Línea Base Operativa (Baseline) (docker stats, docker logs)
- Fase 3: Conexión y Trazabilidad Interna (Oracle) (sqlplus, SPOOL, SET TIMING ON, LINESIZE, PAGESIZE)
- Fase 4: Telemetría Activa (EXPLAIN PLAN, medir la latencia)
- Fase 5: Intervención (Aplica la mejora, p.eje: créate index)
- Fase 6: Validación Matemática (Demuestra que la intervención funcionó: EXPLAIN, medir latencia y se apaga SPOOL)
- Fase 7: Documentación extrayendo datos críticos, Consolidación (git add .) y Commit (git commit)

1.6.2 Documentación como Código: Markdown y GitOps

Registro de Intervención: Optimización de Búsqueda de Empleados
Autor: [Nombre del Estudiante]
Fecha: [Fecha actual]
Archivos de evidencia adjuntos: `01_baseline.txt`, `03_evidencia_oracle.log`

1. Objetivo de la Intervención
Reducir el tiempo de respuesta de las consultas sobre la tabla `tabla_auditoria` filtrando por el nombre del objeto.

2. Línea Base Operativa (Contexto del Sistema)
Antes de la intervención, el servidor se encontraba en condiciones normales:
- **Consumo RAM anfitrión:** 4.2 GB usados de 16 GB libres.
- **Anomalías en logs:** Ninguna reportada en `dmesg` ni en `alert.log`.

3. Diagnóstico Inicial (El "Antes")
Se analizó el plan de ejecución de la consulta original.
- **Operación detectada:** `TABLE ACCESS FULL` (Escaneo completo de la tabla).
- **Costo algorítmico estimado:** 412.
- **Latencia empírica (Tiempo real):** 00:00:02.45 (2.45 segundos).

4. Intervención Realizada
Se procedió a crear un índice tipo B-Tree sobre la columna de búsqueda para evitar el escaneo secuencial.
Comando ejecutado:
\`\`\`sql
CREATE INDEX idx_objeto_nombre ON tabla_auditoria(object_name);
\`\`\`
(Se reportó un error tipográfico inicial `ORA-00942` corregido en el segundo intento, ver log para detalle).

5. Validación de Resultados (El "Después")
Se volvió a ejecutar la misma consulta tras crear el índice, obteniendo:
- **Nueva Operación:** `INDEX RANGE SCAN`.
- **Nuevo Costo algorítmico:** 3.
- **Nueva Latencia empírica:** 00:00:00.02 (0.02 segundos).

6. Conclusión
La intervención fue un éxito. Se logró reducir el tiempo de respuesta en un **99%**, pasando de 2.45 segundos a 0.02 segundos, minimizando el impacto en la I/O del disco.

1.6.3 Gobierno Ético y Fideicomiso del Dato

El dato es el activo más valioso de la organización. Si se expone, se convierte instantáneamente en su riesgo más tóxico.

Su **custodia y protección** absolutas **no son negociables**.

El DBA **no confía en la "*buena voluntad*"** para proteger la información.

Diseña arquitecturas donde la exposición indebida de datos es estructuralmente imposible. El tratamiento de la **Información de Identificación Personal** (PII —datos clínicos, financieros, biométricos o identificativos—) **se rige por un marco de gobierno estricto**:

Principio Estratégico	Implementación Técnica de Élite
Arquitectura *Zero Trust*	Aplicación implacable del **Principio de Menor Privilegio** (PoLP). **Nadie, ni siquiera el DBA superior, consulta datos sensibles en producción sin una justificación temporal y criptográficamente auditada** (acceso Just-In-Time).
Enmascaramiento (*Data Masking*)	Ofuscación dinámica y estática obligatoria. **Los entornos de desarrollo**, pruebas o QA jamás contienen PII real. Se **despliegan algoritmos de anonimización** que preservan la integridad referencial y el formato, pero destruyen la confidencialidad original.
Auditoría Inmutable	**El acceso a datos críticos se somete a telemetría extrema**. Se configuran **auditorías a nivel del motor** del SGBD que **disparan alertas automáticas al Centro de Operaciones de Seguridad** (SOC) **ante lecturas masivas anómalas o exportaciones no programadas**.
Cumplimiento como Código	Los **marcos regulatorios internacionales** (como GDPR, PCI-DSS o HIPAA) **no se delegan al departamento jurídico**. **Se traducen en políticas técnicas**: **cifrado en reposo** (TDE), **restricciones de acceso a nivel de fila** (Row-Level Security) y **purgado automatizado por caducidad legal**.

1.6.4 Estandarización Operativa

La ingeniería de datos de primer nivel **penaliza severamente el conocimiento aislado y la improvisación**.

Las operaciones de misión crítica **no pueden depender de la memoria de un administrador individual ni de intervenciones heroicas durante una crisis**.

Las organizaciones de élite **codifican su experiencia mediante la documentación reproducible**, transformando el talento individual en una **capacidad institucional auditable**.

Para **erradicar la variabilidad**, reducir los errores humanos y asegurar tiempos de resolución predecibles (MTTR), los equipos de operaciones y bases de datos **se rigen estrictamente por dos artefactos fundamentales**:

Runbooks (Operación Rutinaria)

Son procedimientos deterministas y secuenciales para ejecutar tareas predecibles. Incluyen instrucciones exactas y comandos validados para realizar backups, parcheos programados, rotación de credenciales, purgado de logs o validación de estado. Por ello se crea un repositorio de runbooks.

Playbooks (Respuesta a Incidentes)

Árboles de decisión estratégicos diseñados para el caos.

Definen cómo diagnosticar, contener y mitigar de forma segura las interrupciones del servicio, caídas de rendimiento o corrupción de datos bajo presión extrema. Por ello se crea un repositorio de playbooks.

1.6.4 Cultura Kaizen y Evolución Continua del Ecosistema

Un sistema de bases de datos de misión crítica **nunca es un producto terminado**, sino un **ecosistema vivo sometido a una entropía constante**.

A medida que crecen los volúmenes de información,

- ✓ **mutan las amenazas de seguridad y**
- ✓ **cambian las exigencias del negocio**,
- ✓ cualquier **configuración estática se vuelve inevitablemente obsoleta**.

El profesional de élite abandona la complacencia del "***si funciona, no lo toques***" y adopta un ciclo iterativo riguroso: **Observar, Medir, Refinar y Automatizar**.

Esta madurez operativa se manifiesta en **tres ejes fundamentales**:

1. Evolución Arquitectónica (De lo artesanal a lo sistémico)

Todo proceso manual es un candidato a la automatización. La evolución natural de un entorno local directamente sobre el sistema operativo, sin dockerizar, ilustra perfectamente este ciclo.

El trabajo comienza con una instalación manual, pero la mejora continua exige evolucionar hacia el ***scripting automatizado de respaldos, la captura de métricas del kernel*** y, eventualmente, la ***codificación de toda esa infraestructura para que sea replicable en la nube***.

1.6.4 Cultura Kaizen y Evolución Continua del Ecosistema

2. Gestión del Error como Telemetría

En la alta ingeniería, ***los incidentes y las caídas de rendimiento no son fracasos***; son "***telemetría no planificada***".

Cada cuello de botella o bloqueo de transacciones revela los límites matemáticos y físicos de la arquitectura actual, proporcionando la evidencia empírica necesaria para rediseñar los índices, ajustar la memoria o refinar el modelo de datos.

3. Vigilancia Tecnológica Activa

El universo del **almacenamiento muta con extrema rapidez**.

El ingeniero debe evaluar continuamente si **la topología actual sigue siendo la respuesta óptima** o si las **nuevas cargas** operativas **justifican la integración** de **bases de datos vectoriales, columnares o motores autónomos**.

Formar esta mentalidad de evolución perpetua garantiza que el estudiante no solo aprenda a **operar las herramientas del presente**, sino que desarrolle el **criterio técnico**, la **resiliencia** y la **visión arquitectónica** necesarias para liderar la infraestructura del futuro.

UCAM

Gracias por su atención.

1.5.2 Topología de Cargas de Trabajo: OLTP, OLAP y HTAP

Teorema de CAP: Consistency, Availability, Partition Tolerance

Formulado por Eric Brewer, es un principio fundamental en el diseño de bases de datos y sistemas distribuidos.

Establece que ***es imposible que un sistema de datos distribuido garantice simultáneamente las tres propiedades***:

- **Consistencia** (C): Todos los nodos (servidores) del sistema ven exactamente los mismos datos al mismo tiempo. Si un usuario actualiza un registro y acto seguido otro usuario lo lee, el sistema garantiza que la lectura devolverá el dato más reciente.
- **Disponibilidad** (A): Cada petición (lectura o escritura) que se hace al sistema recibe una respuesta exitosa, sin importar si un nodo individual se ha caído. El sistema nunca responde con un error de "fuera de servicio", aunque la respuesta podría no contener el dato más reciente.
- **Tolerancia a Particiones**: El sistema sigue funcionando incluso si la red falla y los nodos pierden la comunicación entre ellos (se crea una "partición" en la red donde un grupo de servidores no puede hablar con el otro).

Un **motor ejecutándose de manera nativa en un ordenador**, sin clústeres ni redes distribuidas de por medio, es un sistema que por naturaleza no sufre particiones de red.

Goza de consistencia y disponibilidad inmediatas de forma local, pero si el disco físico falla o el sistema operativo se bloquea, la base de datos entera cae.

1.5.2 Topología de Cargas de Trabajo: OLTP, OLAP y HTAP

Teorema de CAP: Consistency, Availability, Partition Tolerance

Los **sistemas distribuidos** a nivel corporativo asumen **que las particiones de red (P) ocurrirán y la tolerancia a particiones no es opcional**. Las redes fallan, los cables se rompen y los routers se saturan. Por lo tanto, cuando ocurre una partición de red, el arquitecto de datos debe tomar una decisión drástica y elegir entre:

- **Sistemas CP (Consistencia + Tolerancia a Particiones):** Si la red se corta, el sistema prefiere rechazar peticiones (sacrifica la disponibilidad) antes que entregar datos desactualizados.
    - Ejemplo: Bases de datos como MongoDB o HBase. Si un cajero automático pierde conexión con el servidor central, es mejor que muestre un error a que te permita retirar dinero de un saldo desactualizado.
- **Sistemas AP (Disponibilidad + Tolerancia a Particiones):** Si la red se corta, los nodos siguen respondiendo a las peticiones con los datos que tienen a mano, aunque estén desactualizados (sacrifica la consistencia estricta).
    - Ejemplo: Bases de datos como Cassandra o DynamoDB. Si el sistema de "Me gusta" de una red social pierde sincronización, es mejor dejar que los usuarios sigan viendo y dando likes (alta disponibilidad), aunque temporalmente vean una cantidad de likes distinta a la de sus amigos.

Históricamente, los **sistemas Relacionales distribuidos** tienden a **priorizar la Consistencia** (CP): si un nodo pierde conexión con el primario, rechaza escrituras para no corromper la verdad global, sacrificando la disponibilidad.

Muchos **sistemas NoSQL** nativos de la nube **priorizan la Disponibilidad** (AP): permiten que los nodos desconectados sigan aceptando escrituras, resolviendo los conflictos más tarde (lo que se conoce como ***Consistencia Eventual***).

Establece que es imposible que un sistema de datos distribuido garantice simultáneamente las tres propiedades.

Es decir, un sistema de datos distribuido, ante un fallo de red o desconexión entre los nodos (Tolerancia a Particiones o P),